GPU Howto Recipes MPI NVHPC cuda cuda aware mpi

Example of a Minimal Program Using the NVHPC Stack with CUDA-aware MPI

About NVHPC

NVHPC is an integrated collection of software tools and libraries distributed by Nvidia. An overview document describing NVHPC can be found here. The aim of the NVHPC team is to provide an up-to-date and preconfigured suites of compilers, libraries, and tools that are specifically optimized for Nvidia GPU hardware. It supports both single and multi-GPU execution.

Basic Usage Example

This example shows steps for using NVHPC to run a simple test MPI program, written in C, that communicates between two GPUs across two MPI tasks. MPI requires that the communication between GPUs goes through CPUs. For the much faster communication directly between GPUs, you can use NCCL. The detailed steps that can be executed in an interactive Slurm session are explained below. A complete Slurm job example is shown at the end.

1. Activate the Relevant NVHPC Module

The NVHPC environment is installed as a module and can be made visible in a session using the command

```
module load nvhpc/24.5
```

This will add a specific version of the NVHPC software (version 24.5 on Engaging was released in 2024) to a shell or batch script. The software added includes compilers for C, C++ and Fortran; base GPU optimized numerical libraries for linear algebra, Fourier transforms and others; and GPU optimized communication libraries supporting MPI, SHMEM and NCCL APIs.

An environment variable, `NVHPC_ROOT`, is also set. This can be used in scripts to reference the locations of libraries when needed.

2. Set paths needed for compile step

Here we use the module environment variable, `NVHPC_ROOT`, to set environment variables that have paths needed for compilation and linking of code.

```
culibdir=$NVHPC_ROOT/cuda/lib64
cuincdir=$NVHPC_ROOT/cuda/include
```

3. Create a C program that executes some simple multi-task, multi-GPU test code

The next step is to create a file holding C code that uses MPI to send information between two GPUs running in different processes. Paste the C code below into a file called `test.c`.

test.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>
#include <cuda_runtime.h>
int main(int argc, char *argv[])
{
  int myrank, mpi_nranks;
  int LBUF=1000000;
  float *sBuf_h, *rBuf_h;
  float *sBuf_d, *rBuf_d;
  int bSize;
  int i;

  MPI_Init(&argc, &argv);
  MPI_Comm_rank(MPI_COMM_WORLD, &myrank);                  // my MPI rank
  MPI_Comm_size(MPI_COMM_WORLD, &mpi_nranks);

  if ( mpi_nranks != 2 ) { printf("Program requires exactly 2 ranks\n");exit(-1);
}

  int deviceCount;
  cudaGetDeviceCount(&deviceCount);               // How many GPUs?
  printf("Number of GPUs found = %d\n",deviceCount);
  int device_id = myrank % deviceCount;
  cudaSetDevice(device_id);                       // Map MPI-process to a GPU
  printf("Assigned GPU %d to MPI rank %d of %d.\n",device_id, myrank,
mpi_nranks);

  // Allocate buffers on each host and device
  bSize = sizeof(float)*LBUF;
  sBuf_h = malloc(bSize);
  rBuf_h = malloc(bSize);
  for (i=0;i<LBUF;++i){
    sBuf_h[i]=(float)myrank;
    rBuf_h[i]=-1.;
  }
  if ( myrank == 0 ) {
```

```
    printf("rBuf_h[0] = %f\n",rBuf_h[0]);
   }

   cudaMalloc((void **)&sBuf_d,bSize);
   cudaMalloc((void **)&rBuf_d,bSize);

   cudaMemcpy(sBuf_d,sBuf_h,bSize,cudaMemcpyHostToDevice);

   if ( myrank == 0 ) {
    MPI_Recv(rBuf_d,LBUF,MPI_REAL,1,0,MPI_COMM_WORLD,MPI_STATUS_IGNORE);
   }
   else if ( myrank == 1 ) {
    MPI_Send(sBuf_d,LBUF,MPI_REAL,0,0,MPI_COMM_WORLD);
   }
   else
   {
    printf("Unexpected myrank value %d\n",myrank);
    exit(-1);
   }

   cudaMemcpy(rBuf_h,rBuf_d,bSize,cudaMemcpyDeviceToHost);
   if ( myrank == 0 ) {
    printf("rBuf_h[0] = %f\n",rBuf_h[0]);
   }

   MPI_Barrier(MPI_COMM_WORLD);
   MPI_Finalize();
}
```

4. Compile program

Here we use the NVHPC MPI wrapper to compile. The two environment variables we set earlier (`cuincdir` and `culibdir`) are used to let the compile step know where to find the relevant CUDA header and library files. The CUDA runtime library (`cudart`) is added as a location for finding CUDA functions the code utilizes.

```
mpicc test.c -I${cuincdir} -L${culibdir} -lcudart
```

5. Execute program

Once the code has been compiled, the `mpiexec` command that is part of the `nvhpc` module can be used to run the test program. The `nvhpc` module defaults to using its builtin version of OpenMPI. The OpenMPI option `btl_openib_warn_no_device_params_found` is passed into the OpenMPI runtime library. This option suppresses a warning that OpenMPI can generate when it encounters a network device card that is not present in a built-in list that OpenMPI has historically included.

```
salloc -n 2 --gres=gpu:2
```

```
mpiexec --mca btl_openib_warn_no_device_params_found 0 -n 2 ./a.out
```

Note the `salloc` command is only needed to run interactively from the login node. If you are running in a batch job or are already in an interactive job on a compute node you will not need to first run `salloc`.

Running this program using the command above should produce the following output, although some of the lines may be in different orders due to the asynchronous nature of the tasks.

```
Number of GPUs found = 1
Number of GPUs found = 1
Assigned GPU 0 to MPI rank 0 of 2.
rBuf_h[0] = -1.000000
Assigned GPU 0 to MPI rank 1 of 2.
rBuf_h[0] = 1.000000
```

Example of Slurm job file for executing this example

First create a file called "test.c" containing the example C program above. The job script file below will run all the steps described above for "test.c". It can be submitted to Slurm using the command `sbatch` followed by the filename holding the job script. The output will be written to a file named myjob.log-jobID where jobID is the ID of the specific job you ran.

test_cuda_and_mpi.sbatch

```
#!/bin/bash
#SBATCH -n 2
#SBATCH --gres=gpu:2
#SBATCH -o myjob.log-%A
#
# Basic slurm job that tests GPU aware MPI in the NVHPC tool stack.
#
#
#   To submit through Slurm use:
#
#   $ sbatch test_cuda_and_mpi.sbatch
#
#   in terminal.

# Write a little log info
echo "## Start time \""`date`"\""
echo "## Slurm job running on nodes \"${SLURM_JOB_NODELIST}\""
echo "## Slurm submit directory \"${SLURM_SUBMIT_DIR}\""
echo "## Slurm submit host \"${SLURM_SUBMIT_HOST}\""
echo " "
```

```
module load nvhpc/24.5
culibdir=$NVHPC_ROOT/cuda/lib64
cuincdir=$NVHPC_ROOT/cuda/include

mpicc test.c -I${cuincdir} -L${culibdir} -lcudart

mpiexec --mca btl_openib_warn_no_device_params_found 0 -n 2 ./a.out
```

September 5, 2025